МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВ РОССИЙСКОЙ ФЕДЕРАЦИИ

федерального государственного бюджетного образовательного учреждения высшего образования
«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ Г.В. ПЛЕХАНОВА»
Пермский институт (филиал)

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к проекту производственной практики ПП.02.01

Тема: «Разработка модуля импорта товаров в электронный магазин из CSV-файлов»

Студент: Паркачев Ратмир Александрович
Группа: ИПо-42
Место практики: ООО «Виджет»
Период практики: 03 ноября 2025 г. — 13 ноября 2025 г.

Пермь, 2026 г.

Оглавление

Введение 3
Глава 1. Технологический стек и архитектура проекта 4
1.1 Используемые технологии и инструменты 4
1.2 Архитектура приложения 4
Глава 2. Модель данных и хранение товаров 5
2.1 Обзор моделей 5
2.2 Репозиторий в памяти 6
Глава 3. Сервис импорта CSV 7
3.1 Парсер CSV 7
3.2 Валидация и результат импорта 8
Глава 4. MVC-интерфейс 9
4.1 ProductsController 9
4.2 Представления Import и Index 10
Глава 5. Безопасность и рабочий процесс 11
5.1 Загрузка файла 11
5.2 Последовательность импорта 12
Глава 6. Модульное тестирование 13
6.1 Структура тестового проекта 13
6.2 Результаты тестирования 14
Глава 7. Ограничения и качество решения 15
Заключение 16

Введение

Настоящая пояснительная записка посвящена разработке ASP.NET Core MVC-модуля, предназначенного для наполнения электронного магазина товарами путём чтения данных из CSV-файлов. Корректные записи преобразуются в объекты C# и сохраняются в потокобезопасном репозитории в памяти приложения.

Работа выполнена в рамках производственной практики ПП.02.01 «Осуществление интеграции программных модулей» по варианту предметной области «Электроника». Пользователь может загрузить CSV-файл через браузер, получить результат импорта с количеством обработанных строк и перечнем ошибок, а также просмотреть загруженные товары в таблице.

Цель работы — получить практический опыт анализа требований, проектирования сервисного слоя, интеграции программного модуля в ASP.NET Core MVC-приложение, тестирования и проверки качества кода.

Для достижения цели были решены следующие задачи:

- определён формат CSV и правила валидации товаров электроники;
- реализованы модель Product и результат импорта;
- созданы CSV-парсер и сервис импорта с обработкой ошибок;
- реализован thread-safe in-memory repository, зарегистрированный через DI как Singleton;
- созданы страницы загрузки CSV и просмотра списка товаров;
- разработаны автоматические тесты xUnit и выполнена проверка сборки.

Глава 1. Технологический стек и архитектура проекта

1.1 Используемые технологии и инструменты

Для реализации выбран стек, соответствующий заданию производственной практики. Приложение не использует базу данных, ORM или внешние API: товары хранятся только в памяти текущего процесса.

Компонент	**Технология / решение**
Платформа	.NET 10.0
Язык программирования	C#
Web-фреймворк	ASP.NET Core MVC
Представления	Razor Views
Хранение	List<Product> в потокобезопасном Singleton-репозитории
Импорт	CSV из загруженного IFormFile
Тестирование	xUnit; 48 автоматических тестов
Контроль версий	Git

1.2 Архитектура приложения

Архитектура построена с разделением ответственности. Модели описывают данные, сервисный слой выполняет разбор и бизнес-валидацию, репозиторий отвечает за хранение, а контроллер связывает HTTP-запросы с представлениями и сервисами.

```
CSV-файл
↓
ProductsController
↓
IProductImportService / ProductImportService
├── CsvLineParser
└── IProductRepository / InMemoryProductRepository
↓
ImportResult → Razor View
```

Глава 2. Модель данных и хранение товаров

2.1 Обзор моделей

Основной сущностью приложения является Product. Модель содержит базовые характеристики товара и дополнительные поля варианта «Электроника». Результат импорта представлен классом ImportResult, а отдельная ошибка — классом ImportError.

Класс	Назначение
Product	Артикул, название, описание, цена, валюта, категория, остаток, изображение, активность и характеристики электроники.
ImportResult	Общее количество строк, число импортированных товаров, число ошибок и список ошибок.
ImportError	Физический номер строки и понятное сообщение об ошибке.
ImportViewModel	Имя загруженного файла и результат импорта для страницы Import.
ProductsIndexViewModel	Список товаров, количество и общая стоимость для страницы Index.

2.2 Поля товара

Поле	Тип	Правило
Sku	string	Обязательно, уникально без учёта регистра и пробелов.
Name	string	Обязательно.
Description	string	Может быть пустым.
Price	decimal	Число строго больше 0.
Currency	string	Пустое значение нормализуется в RUB.
CategoryName	string	Пустое значение нормализуется в «Без категории».
StockQuantity	int	Целое число не меньше 0.
IsActive	bool	true, false, 1 или 0.
Brand	string	Обязательно для электроники.
WarrantyMonths	int	Целое число от 0 до 60.
ModelNumber	string	Может быть пустым.

Глава 2. Модель данных и хранение товаров

2.3 Репозиторий в памяти

Интерфейс IProductRepository определяет операции Add, TryAdd, GetAll, GetBySku, Count и Clear. Реализация InMemoryProductRepository использует List<Product> и объект блокировки. Каждая операция, работающая с коллекцией, выполняется внутри lock.

Метод GetAll возвращает копию списка, поэтому внешний код не получает ссылку на внутреннюю изменяемую коллекцию. Поиск по SKU выполняется после Trim и без учёта регистра. Метод Add проверяет null и выбрасывает понятный ArgumentNullException.

2.4 Атомарное добавление

Для предотвращения дубликатов при параллельных запросах добавлен метод TryAdd. Проверка существования SKU и добавление товара выполняются в одной критической секции. Если такой SKU уже присутствует, метод возвращает false и не меняет коллекцию.

```
public bool TryAdd(Product product)
{
lock (_lock)
{
if (duplicateSku) return false;
_products.Add(product);
return true;
}
}
```

Через контейнер зависимостей репозиторий зарегистрирован как Singleton. Это обеспечивает общее хранилище между HTTP-запросами в течение работы приложения.

Глава 3. Сервис импорта CSV

3.1 Парсер CSV

CsvLineParser.Parse разбирает одну физическую строку CSV. Поддерживаются запятые-разделители, пустые поля, значения в двойных кавычках, запятые внутри кавычек и экранированные кавычки через удвоение. После завершения поля выполняется Trim.

Парсер является упрощённым однострочным решением. Поля, содержащие переводы строк, и полный набор правил RFC 4180 в учебном проекте не реализуются, поскольку импорт выполняется построчно.

Сценарий	Ожидаемое поведение
Обычная строка	Поля разделяются запятыми.
Пустое поле	Сохраняется как пустая строка.
Запятая в кавычках	Не создаёт дополнительное поле.
Экранированная кавычка	Две кавычки преобразуются в одну.
Завершающее пустое поле	Последнее поле не теряется.
Пустая строка	Возвращается пустой список.

3.2 Заголовок и колонки

Сервис находит первую непустую строку как заголовок, запоминает её физический номер и проверяет наличие двенадцати обязательных колонок. Сравнение заголовков выполняется без учёта регистра и внешних пробелов. Дополнительные колонки допускаются, а при повторяющемся имени используется первое вхождение.

Глава 3. Сервис импорта CSV

3.3 Валидация и результат импорта

ProductImportService предоставляет ImportFromCsv для файлового сценария и ImportFromLines для тестирования без зависимости от файловой системы. В ImportFromCsv проверяются путь и существование файла, затем обрабатываются ошибки чтения и строки передаются в общую логику импорта.

Для каждой непустой строки увеличивается TotalRows. Сервис проверяет количество полей, затем последовательно валидирует sku, name, price, stock, is_active, brand и warranty_months. Все ошибки одной строки собираются до принятия решения о добавлении товара.

Этап	Результат
Ошибки файла	Ошибка с RowNumber = 0.
Пустые данные	Сообщение «Данные для импорта отсутствуют».
Ошибка заголовка	Ошибка с физическим номером заголовка.
Ошибка строки	Ошибка с физическим номером строки данных.
Ошибки валидации	Все ошибки строки добавляются в результат.
Дубликат SKU	Товар не добавляется, добавляется ошибка.
Корректная строка	Создаётся Product и увеличивается ImportedCount.

3.4 Нормализация данных

Пустая валюта заменяется на RUB, пустая категория — на «Без категории». Строковые значения очищаются от внешних пробелов. Логические значения true и false поддерживаются без учёта регистра, а также в числовом виде 1 и 0.

```
ImportResult
TotalRows — обработанные непустые строки
ImportedCount — успешно добавленные товары
ErrorCount — Errors.Count
Errors — номер строки и сообщение
```

Глава 4. MVC-интерфейс

4.1 ProductsController

ProductsController содержит три пользовательских действия: Index, GET Import и POST Import. Контроллер получает IProductRepository и IProductImportService через конструктор. CSV-парсинг и проверка полей не находятся в контроллере.

Действие	Назначение
Index()	Получает снапшот товаров и передаёт ProductsIndexViewModel в представление.
Import() GET	Возвращает пустую модель формы загрузки.
Import(IFormFile?) POST	Проверяет файл, передаёт строки сервису и показывает результат.

4.2 Dependency Injection

```
builder.Services.AddSingleton<IProductRepository, InMemoryProductRepository>();
builder.Services.AddSingleton<IProductImportService, ProductImportService>();
```

Singleton выбран только для in-memory repository и сервиса импорта. Такой жизненный цикл соответствует задаче: товары должны быть доступны между запросами, но после перезапуска приложения могут быть потеряны.

4.3 Защита POST-запроса

POST-действие помечено ValidateAntiForgeryToken, а форма содержит @Html.AntiForgeryToken(). Это снижает риск CSRF-атак при отправке формы импорта.

Глава 4. MVC-интерфейс

4.4 Представление Import

Страница импорта содержит input типа file с multipart/form-data и ограничением accept=.csv. После загрузки отображаются TotalRows, ImportedCount и ErrorCount. При наличии ошибок формируется таблица с физическим номером строки и описанием проблемы.

4.5 Представление Index

Страница списка товаров выводит таблицу с SKU, названием, описанием, ценой, валютой, категорией, остатком, активностью, брендом, гарантией и моделью. Если репозиторий пуст, пользователь видит понятное сообщение о том, что товары ещё не импортированы.

Страница	Основные элементы
Главная	Описание приложения и переходы к импорту и списку.
Import	Выбор CSV, запуск импорта, сводка и таблица ошибок.
Index	Таблица импортированных товаров и переход к импорту.
Error	Русскоязычное сообщение о непредвиденной ошибке.

4.6 Локализация

Видимый интерфейс переведён на русский язык. В корневом HTML установлен атрибут lang="ru"; пункты навигации, страницы Privacy и Error используют русские подписи.

Глава 5. Безопасность и рабочий процесс

5.1 Загрузка файла через браузер

Пользователь не вводит произвольный серверный путь. Контроллер принимает IFormFile, очищает имя через Path.GetFileName и работает с потоком загруженного файла. Файл не сохраняется в постоянное хранилище и не используется как путь, переданный пользователем.

Проверка	**Реализация**
Расширение	Разрешается .csv без учёта регистра, включая .CSV.
Размер файла	Максимум 5 МБ; превышение отклоняется с понятным сообщением.
HTTP-запрос	Для действия установлен лимит запроса 6 МБ.
Имя файла	Применяется Path.GetFileName.
Ошибки	Показываются в модели результата без падения приложения.
CSRF	POST защищён anti-forgery token.

5.2 Ограничения

Данные хранятся только в памяти процесса. После остановки или перезапуска приложения список товаров очищается. Реализованный CSV-парсер работает с физическими строками и не поддерживает многострочные поля в кавычках.

5.3 Повторный импорт

Повторный запуск импорта не приводит к падению приложения. Корректные новые товары добавляются, а уже существующие SKU пропускаются с ошибкой. При параллельных запросах атомарный TryAdd не допускает добавления двух товаров с одним SKU.

Глава 5. Безопасность и рабочий процесс

5.4 Рабочий процесс пользователя

- пользователь открывает страницу /Products/Import;
- выбирает CSV-файл с компьютера;
- сервер проверяет наличие файла, расширение и размер;
- строки читаются в UTF-8 и передаются ProductImportService;
- сервис формирует результат с количеством строк, импортов и ошибок;
- пользователь просматривает ошибки и переходит к списку товаров.

5.5 Пример корректного CSV

```
sku,name,description,price,currency,category_name,stock,image_url,is_active,brand,warranty_months,model_number
1001,Ноутбук Lenovo IdeaPad 3,Бюджетный
ноутбук,45990,RUB,Компьютеры,5,/images/laptop.jpg,true,Lenovo,24,IdeaPad3-15
```

5.6 Тестовые файлы

Для проверки подготовлены UTF-8-файлы valid_electronics.csv, invalid_price.csv, duplicate_sku.csv, missing_brand.csv, warranty_out_of_range.csv, bad_active.csv и empty_file.csv. В wwwroot/csv/sample_products.csv размещён пример с 11 товарами электроники.

Глава 6. Модульное тестирование

6.1 Структура тестового проекта

Тестовый проект ECommerceCsvImport.Tests использует xUnit и ссылается на основной ASP.NET Core MVC-проект. Тесты разделены по ответственности: репозиторий, CSV-парсер, сервис импорта и контроллер.

Тестовый класс	Кол-во	Основные сценарии
RepositoryTests	6	Добавление, поиск, регистронезависимый SKU, пустой репозиторий, атомарный и конкурентный TryAdd.
CsvLineParserTests	6	Простая строка, пустые поля, кавычки, запятая внутри кавычек, экранирование, завершающее поле.
ProductImportServiceTests	31	Успешный импорт, ошибки файла и заголовка, валидация, дубликаты, нормализация, номера строк.
ProductsControllerTests	5	Успешная загрузка, расширение, размер, пустой файл и .CSV.

6.2 Базовый план и расширение

В плане выполнения индивидуального задания указаны 31 базовый автоматический тест: 4 для репозитория, 4 для парсера и 23 для сервиса. Также в PDF перечислены 5 ручных UI-сценариев. В фактическом проекте набор расширен до 48 автоматических тестов для снижения риска необнаруженных ошибок.

Дополнительные проверки включают атомарность и конкурентность репозитория, сохранение завершающего пустого поля, физические номера строк, нулевую цену, нечисловые значения, границы гарантии, регистронезависимые true/false и тесты контроллера.

Глава 6. Модульное тестирование

6.3 Проверенные сценарии сервиса

Группа	Проверки
Успешные данные	Одна и несколько строк, кириллица, пустые необязательные поля.
Файл и заголовок	Несуществующий/пустой файл, только заголовок, неверные обязательные колонки.
Структура строки	Неверное количество полей, пустые строки, физические номера.
Обязательные поля	Пустые sku, name и brand, включая значения из пробелов.
Числа	Отрицательная/нулевая/нечисловая цена, отрицательный и нечисловой stock.
Логика	Недопустимый is_active, true/false/1/0 без учёта регистра.
Гарантия	Меньше 0, больше 60, значения 0 и 60.
Дубликаты	Внутри файла, в заполненном репозитории, параллельное добавление.
Нормализация	Валюта RUB, категория «Без категории».

6.4 Результат запуска

```
dotnet test ECommerceCsvImport.sln --no-restore
Passed: 48
Failed: 0
Skipped: 0
Total: 48
```

Дополнительно выполнена Release-сборка решения. Она завершилась без предупреждений и ошибок. Проверка git diff --check также завершилась успешно.

6.5 Ручные UI-сценарии

Через локальный HTTP-интерфейс проверены успешный импорт, неверная цена, дубликат SKU, пустой файл и неверное значение is_active. Для загрузки используется браузерная multipart-форма.

Глава 7. Ограничения и качество решения

7.1 Разделение ответственности

Контроллер выполняет только HTTP-операции и подготовку модели представления. Разбор CSV, нормализация и валидация находятся в ProductImportService. InMemoryProductRepository инкапсулирует коллекцию товаров и синхронизацию доступа.

7.2 Проверка требований качества

Критерий	Результат
Понятные имена	Классы и методы отражают назначение.
Обработка ошибок	Ошибки CSV не приводят к падению приложения.
Безопасность	Путь сервера не принимается; проверяются расширение и размер.
Потокобезопасность	Коллекция защищена lock; TryAdd атомарен.
Тестируемость	ImportFromLines позволяет тестировать без файловой системы.
Читаемость	Основная бизнес-логика разделена по слоям.
Документация	README, отчёт, дневник, доклад и пояснительная записка обновлены.

7.3 Возможные улучшения

В дальнейшем проект можно расширить постоянным хранилищем, поиском и фильтрацией товаров, сортировкой, пагинацией, журналированием и экспортом. Эти функции не включены в обязательный объём задания, поскольку текущая реализация должна использовать in-memory-хранилище.

Заключение

В рамках производственной практики разработан ASP.NET Core MVC-модуль импорта товаров в электронный магазин из CSV-файлов. Реализованы модель Product с характеристиками электроники, парсер CSV, сервис импорта и потокобезопасный Singleton-репозиторий в памяти.

Веб-интерфейс предоставляет форму загрузки файла, показывает статистику импорта и список ошибок, а также отображает загруженные товары в таблице. В контроллере отсутствует сложная бизнес-логика: она вынесена в сервис и репозиторий.

Для повышения надёжности реализован расширенный набор из 48 автоматических тестов xUnit. Проверены положительные и отрицательные сценарии, граничные значения, кавычки CSV, дубликаты SKU, конкурентный доступ, безопасность загрузки и работа контроллера. Все тесты пройдены успешно, Release-сборка завершилась без предупреждений и ошибок.

Поставленная цель достигнута: создан работоспособный учебный модуль, соответствующий требованиям производственной практики и подготовленный к демонстрации и защите.

Список использованных материалов

- План выполнения индивидуального задания ПП.02.01.
- РЭУ 4 курс Производственная практика ПП.02.01.
- Исходный код проекта ECommerceCsvImport.
- Документация ASP.NET Core MVC и xUnit.